动态开点线段树

2026.4.22，在补25CCPC重庆J，思路一眼秒了，但是哥们怎么好像不会写动态开点的线段树？

CF Edu36 E：Problem - 915E - Codeforces

2024CCPC济南L：The Tower - Problem - QOJ.ac

2025CCPC重庆J：Tetris - Problem - QOJ.ac

动态开点线段树

过去的线段树都是处理 $[1,n]$ 的范围，n 一般都是 10^5 的大小，但是如果要处理 $[1,10^9]$ 的范围，甚至是负数的范围，就需要动态开点的线段树

过去的线段树是直接把长度为 n 的线段树全部创建出来，而动态开点是，等我们需要我们才创建，类似于线段树。比如现在有一个节点表示区间为 $[10,20]$ ，需要修改 $[16,18]$ 的信息时，就创建表示 $[16,20]$ 的线段树节点，然后递归调用，可能全部操作下来，都没有创建过表示区间 $[10,15]$ 的线段树节点

由于空间是动态开的，所以不会有 $lson=2*rt$ ，$rson=2*rt+1$ 这个性质，需要记录一下。为了方便调用，一般会用上宏

```
#define ls(x) (tr[x].l)
#define rs(x) (tr[x].r)
```

还有需要注意的是，mid 的计算方式变成了 $mid=(l+r-1)/2$ ，这样可以处理 l , r 都是负数的情况

关于空间复杂度，用 V 表示值域，每一次操作最多消耗 $log(V)$ 的空间，m 次操作为 $m\log(V)$ ，一般情况下，都是在 $[-10^9,10^9]$ 的范围上操作，$log(V)=30$ 。不过空间复杂度一般都是 $O(\text{能开多大开多大})$

区间加、区间查询

```
struct segment_tree {
    #define ls(x) (tr[x].lson)
    #define rs(x) (tr[x].rson)
    #define len(x) (tr[x].r - tr[x].l + 1)

    struct node {
        int l, r;
        int sum, lazy;
        int lson, rson;
        node() : l(0), r(0), sum(0), lazy(0), lson(0), rson(0) {}
```

```
        node(int l, int r) : l(l), r(r), sum(0), lazy(0), lson(0), rson(0) {}
    };
    vector<node> tr;

    int NewNode(int l = 0, int r = 0) {
        tr.emplace_back(l, r);
        return tr.size() - 1;
    }

    segment_tree (int l, int r) {
        tr.clear();
        tr.reserve(500010);
        NewNode(l, r);
    }

    void pushup(int u) {
        tr[u].sum = tr[ls(u)].sum + tr[rs(u)].sum;
    }

    void pushdown(int u) {
        int l = tr[u].l, r = tr[u].r;
        int mid = (l + r - 1) / 2;
        if(!ls(u)) ls(u) = NewNode(l, mid);
        if(!rs(u)) rs(u) = NewNode(mid + 1, r);
        if(tr[u].lazy) {
            tr[ls(u)].sum += tr[u].lazy * len(ls(u));
            tr[rs(u)].sum += tr[u].lazy * len(rs(u));
            tr[ls(u)].lazy += tr[u].lazy;
            tr[rs(u)].lazy += tr[u].lazy;
            tr[u].lazy = 0;
        }
    }

    int query(int u, int l, int r) {
        if(l <= tr[u].l && tr[u].r <= r) return tr[u].sum;
        if(l > tr[u].r || r < tr[u].l) return 0;
        pushdown(u);
        int mid = (tr[u].l + tr[u].r - 1) / 2;
        return query(ls(u), l, r) + query(rs(u), l, r);
    }

    void modify(int u, int l, int r, int x) {
        if(l <= tr[u].l && tr[u].r <= r) {
            tr[u].sum += len(u) * x;
            tr[u].lazy += x;
            return;
        }
```

```
        if(l > tr[u].r || r < tr[u].l) return;
        pushdown(u);
        int mid = (tr[u].l + tr[u].r - 1) / 2;
        if(l <= mid) modify(ls(u), l, r, x);
        if(r > mid) modify(rs(u), l, r, x);
        pushup(u);
    }
};
```

Physical Education Lessons

题意：从现在到学期结束还有 n 天（编号从 1 到 n），起初所有都是工作日。接下来依次发出 q 条指令，每个指令把一段区间 $[l, r]$ 全部变成工作日或非工作日。每条指令发出后，计算剩余的工作日天数

数据范围：$1 \le n \le 10^9$，$1 \le q \le 3 \cdot 10^5$

```
namespace segment_tree {
    #define ls(x) (tr[x].lson)
    #define rs(x) (tr[x].rson)

    const int N = 15000010;
    struct node {
        int sum, lazy;
        int lson, rson;
    }tr[N];
    int tot;

    int NewNode() {
        tot ++;
        tr[tot] = {0, -1, 0, 0};
        return tot;
    }

    void init(int l, int r) {
        tot = 0;
        tr[0] = {0, -1, 0, 0};
    }

    void pushup(int u) {
        tr[u].sum = tr[ls(u)].sum + tr[rs(u)].sum;
    }

    void pushdown(int u, int l, int r) {
        int mid = (l + r - 1) / 2;
        if(!ls(u)) ls(u) = NewNode();
```

```
        if(!rs(u)) rs(u) = NewNode();
        if(tr[u].lazy == 1) {
            tr[ls(u)].sum = mid - l + 1;
            tr[ls(u)].lazy = 1;
            tr[rs(u)].sum = r - mid;
            tr[rs(u)].lazy = 1;
            tr[u].lazy = -1;
        }
        if(tr[u].lazy == 0) {
            tr[ls(u)].sum = tr[ls(u)].lazy = 0;
            tr[rs(u)].sum = tr[rs(u)].lazy = 0;
            tr[u].lazy = -1;
        }
    }

    int query(int u, int l, int r, int L, int R) {
        if(L <= l && r <= R) return tr[u].sum;
        if(l > R || r < L) return 0;
        pushdown(u, l, r);
        int mid = (l + r - 1) / 2;
        return query(ls(u), l, mid, L, R) + query(rs(u), mid + 1, r, L, R);
    }

    void modify(int u, int l, int r, int L, int R, int c) {
        if(L <= l && r <= R) {
            tr[u].sum = (r - l + 1) * c;
            tr[u].lazy = c;
            return;
        }
        if(l > R || r < L) return;
        pushdown(u, l, r);
        int mid = (l + r - 1) / 2;
        if(L <= mid) modify(ls(u), l, mid, L, R, c);
        if(R > mid) modify(rs(u), mid + 1, r, L, R, c);
        pushup(u);
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    segment_tree::init(1, n);

    while(m --) {
        int op, l, r;
        cin >> l >> r >> op;
        if(op == 1) segment_tree::modify(0, 1, n, l, r, 1);
```

```
        else segment_tree::modify(0, 1, n, l, r, 0);
        cout << n - segment_tree::query(0, 1, n, 1, n) << '\n';
    }
}
```

Tetris

题意：游戏中共有 n 种不同的俄罗斯方块，系统会将每 n 个不同的方块打包成一个 Bag 。在一个 Bag 内，这 n 种方块一定会全部出现一次且仅出现一次（内部顺序完全随机），不同 Bag 之间的顺序完全独立。小 C 记住了游戏中连续掉落的 m 个方块的种类，构成了一个序列 $f_1, f_2, \ldots, f_m$ 。这段记忆是从全局第 $x+1$ 个方块开始的，序列中为 0 的位置表示忘记了具体的方块种类，可以匹配任何方块。求有多少个 x $(0 \le x < n)$ 满足序列完美符合 n-BAG 规则，输出符合的 x 的数量及其异或和。序列还会进行 q 次修改操作

数据范围：$1 \le n \le 10^{18}$ ，$1 \le m \le 5 \times 10^5$ ，$0 \le q \le 5 \times 10^5$ ，$0 \le f_i \le n$

题解：任意两个相同的元素，不能处于同一个 Bag 中，因此合法的 x 是不超过两段连续区间。用 set 维护每种块的下标，线段树维护每个 x 的合法标记数量、合法 x 个数及其异或和。由于 n 为 10^{18} 级别，线段树需要动态开点

```
const int N = 500010;
typedef pair<int,int> pii;

int n, m, q, a[N];
int tot;
map<int,set<int>> s;

struct Segment_Tree {
    #define ls(x) (tr[x].lson)
    #define rs(x) (tr[x].rson)

    int xorsum(int x) {
        if(x < 0) return 0;
        int res = 0;
        for(int i = x; i >= 0; i--) {
            if(i % 4 == 3) break;
            res ^= i;
        }
        return res;
    }
    int xorsum(int l, int r) {
        return xorsum(r) ^ xorsum(l - 1);
    }

    struct node {
```

```
        int l, r;
        int cnt, val, sum, lazy;
        int lson, rson;
        node() : l(0), r(0), cnt(0), val(0), sum(0), lazy(0), lson(0), rson(0) {}
    };
    vector<node> tr;

    int NewNode(int l = 0, int r = 0) {
        tr.emplace_back();
        int pos = tr.size() - 1;
        tr[pos].l = l, tr[pos].r = r, tr[pos].val = xorsum(l, r), tr[pos].sum = r - l + 1;
        return pos;
    }

    void init(int n) {
        tr.clear();
        NewNode(0, n - 1);
    }

    void pushdown(int u) {
        int l = tr[u].l, r = tr[u].r;
        int mid = (l + r - 1) / 2;
        if(!ls(u)) ls(u) = NewNode(l, mid);
        if(!rs(u)) rs(u) = NewNode(mid + 1, r);
        if(tr[u].lazy) {
            tr[ls(u)].cnt += tr[u].lazy;
            tr[ls(u)].lazy += tr[u].lazy;
            tr[rs(u)].cnt += tr[u].lazy;
            tr[rs(u)].lazy += tr[u].lazy;
            tr[u].lazy = 0;
        }
    }

    void pushup(int u) {
        if(tr[ls(u)].cnt == tr[rs(u)].cnt) {
            tr[u].cnt = tr[ls(u)].cnt;
            tr[u].val = tr[ls(u)].val ^ tr[rs(u)].val;
            tr[u].sum = tr[ls(u)].sum + tr[rs(u)].sum;
        } else if(tr[ls(u)].cnt > tr[rs(u)].cnt) {
            tr[u].cnt = tr[ls(u)].cnt;
            tr[u].val = tr[ls(u)].val;
            tr[u].sum = tr[ls(u)].sum;
        } else {
            tr[u].cnt = tr[rs(u)].cnt;
            tr[u].val = tr[rs(u)].val;
```

```
            tr[u].sum = tr[rs(u)].sum;
        }
    }

    void modify(int u, int l, int r, int c) {
        if(l <= tr[u].l && tr[u].r <= r) {
            tr[u].cnt += c;
            tr[u].lazy += c;
            return;
        }
        if(l > tr[u].r || r < tr[u].l) return;
        pushdown(u);
        int mid = (tr[u].l + tr[u].r - 1) / 2;
        if(l <= mid) modify(ls(u), l, r, c);
        if(r > mid) modify(rs(u), l, r, c);
        pushup(u);
    }

    void work() {
        int x, y;
        if(tr[0].cnt != tot) x = 0, y = 0;
        else x = tr[0].sum, y = tr[0].val;
        cout << x << ' ' << y << '\n';
    }
}Tr;

vector<pii> get(int x, int y) {
    vector<pii> res;
    int px = ((n - x) % n + n) % n;
    int py = ((n - y + 1) % n + n) % n;
    if(py <= px) res.push_back({py, px});
    else {
        res.push_back({py, n - 1});
        res.push_back({0, px});
    }
    return res;
}

void add(int x, int y) {
    if(x + n <= y) return;
    auto p = get(x, y);
    tot ++;
    for(auto [l, r]: p)
        Tr.modify(0, l, r, 1);
}

void del(int x, int y) {
```

```
    if(x + n <= y) return;
    auto p = get(x, y);
    tot --;
    for(auto [l, r]: p)
        Tr.modify(0, l, r, -1);
}

void ADD(int pos) {
    int val = a[pos];
    if(val == 0) return;
    auto it = s[val].lower_bound(pos);
    if((it != s[val].end()) && (it != s[val].begin())) {
        auto L = prev(it), R = it;
        int vl = *L, vr = * R;
        del(vl, vr), add(vl, pos), add(pos, vr);
    } else if(it != s[val].begin()) {
        auto L = prev(it);
        int vl = *L;
        add(vl, pos);
    } else if(it != s[val].end()) {
        auto R = it;
        int vr = *R;
        add(pos, vr);
    }
    s[val].insert(pos);
}

void DEL(int pos) {
    int val = a[pos];
    if(val == 0) return;
    auto it = s[val].lower_bound(pos);
    if((it != s[val].begin()) && (next(it) != s[val].end())) {
        auto L = prev(it), R = next(it);
        int vl = *L, vr = * R;
        add(vl, vr), del(vl, pos), del(pos, vr);
    } else if(it != s[val].begin()) {
        auto L = prev(it);
        int vl = *L;
        del(vl, pos);
    } else if(next(it) != s[val].end()) {
        auto R = next(it);
        int vr = *R;
        del(pos, vr);
    }
    s[val].erase(it);
}

```

```
void solve() {
    cin >> n >> m >> q;
    Tr.init(n);
    for(int i = 1; i <= m; i++) {
        cin >> a[i];
        if(a[i]) {
            if(s[a[i]].size()) add(*s[a[i]].rbegin(), i);
            s[a[i]].insert(i);
        }
    }
    Tr.work();
    while(q --) {
        int x, c;
        cin >> x >> c;
        DEL(x);
        a[x] = c;
        ADD(x);
        Tr.work();
    }
}
```